

Sistema de Controle de Pacientes

Consultório Odontológico Dr. Valdomiro Razlosnek Alvares

Especificação Técnica do Projeto

Projeto Integrador — UNIVESP

Engenharia de Computação e Tecnologia da Informação

Eliézer Paes Polsaque · Fabio Kamagian Gomes

Leandro Polsaque · Cleiton Rochel Vieira

São Paulo, 2026

Especificação do Projeto: Sistema de Controle de Pacientes — Consultório Odontológico

1. Visão Geral

Sistema web para gestão de um consultório odontológico, desenvolvido como Projeto Integrador da UNIVESP (Engenharia de Computação e Tecnologia da Informação). O estudo de caso é a clínica do Dr. Valdomiro Razlosnek Alvares, localizada no Tatuapé, SP, onde todos os processos de agendamento, prontuários, financeiro e estoque de materiais são realizados manualmente pela secretária.

O foco central é a **digitalização e centralização** das operações do consultório: agenda, pacientes, prontuários, financeiro, materiais e esterilização — com uma interface clara, voltada principalmente

para uso desktop pela secretária, com responsividade para acesso eventual pelo celular.

2. Decisão de Produto (Arquitetura de Conectividade)

O sistema adota a abordagem **online-first com resiliência de leitura**.

Justificativa: O consultório opera com conexão Wi-Fi fixa, múltiplos usuários simultâneos (secretária + dentista) e dados clínicos sensíveis que não devem ser armazenados no dispositivo local. Um modelo offline-first — onde dados seriam gravados localmente e sincronizados depois — traria complexidade desnecessária de resolução de conflitos e risco de segurança sem benefício real para esse ambiente.

O que acontece quando a internet cai: - O sistema exibe um indicador visual de conectividade. - As telas já carregadas permanecem funcionais (leitura da agenda do dia, dados básicos de pacientes já em memória). - Gravações de novos registros ficam bloqueadas até a reconexão, com mensagem clara para o usuário. - A reconexão é detectada automaticamente e o fluxo normal é retomado sem ação do usuário.

Esse comportamento resolve o cenário mais provável de queda de internet num consultório fixo (consultar quem está agendado, atender o paciente que chegou) sem a complexidade de sincronização bidirecional e sem expor dados clínicos no armazenamento local do browser.

3. Stack Tecnológica

- **Frontend Web:** Angular (framework TypeScript/JavaScript) — SPA (Single Page Application).
 - **Backend:** Java com Spring Boot — API REST.
 - **Banco de Dados:** PostgreSQL (containerizado via Docker).
 - **Controle de Versão:** Git / GitHub.
 - **Deploy:** VPS Hostinger.
-

4. Arquitetura do Sistema

4.1 Frontend (Angular SPA)

- Aplicação de página única servida por servidor HTTP (Nginx em produção).
- Comunicação com o backend exclusivamente via API REST (JSON).
- Cache em memória (Angular Services) para dados de uso frequente: agenda do dia, lista de pacientes, dados do usuário logado. O cache é volátil — existe apenas enquanto a sessão do browser está aberta.
- Módulos organizados por domínio: Atendimento, Financeiro, Materiais, Administração.
- Responsividade via CSS (Bootstrap ou Angular Material) para acesso mobile ocasional.

4.2 Backend (Spring Boot)

- API REST organizada em controllers por domínio (`/api/pacientes` , `/api/agenda` , `/api/financeiro` , etc.).
- Camada de serviço com regras de negócio isoladas dos controllers.
- Spring Security para autenticação e autorização por perfil de usuário.
- Acesso ao banco via Spring Data JPA / Hibernate.

4.3 Banco de Dados (PostgreSQL)

- Todas as entidades usam UUID como chave primária, gerado pelo backend.
- Colunas `created_at` e `updated_at` em todas as tabelas de negócio.
- Soft delete (`deleted_at`) nas entidades principais (Paciente, Consulta, Dentista) para preservar histórico.

5. Telas do Sistema

Com base nos protótipos definidos pelo grupo, o sistema compreende as seguintes telas:

5.1 Geral

- **Login** — autenticação com email/senha.

- **Dashboard** — visão consolidada do dia: consultas agendadas, pacientes ativos, cancelamentos, faltas, receita do mês, valores em aberto.
- **Pendências** — central de tarefas pendentes (confirmações, retornos, cobranças).

5.2 Atendimento

- **Pacientes** — cadastro e busca de pacientes; acesso ao prontuário.
- **Prontuário do Paciente** — histórico de consultas, odontograma, anotações clínicas, documentos.
- **Agenda** — visualização por dia/semana; agendamento de consultas.
- **Grade de Horários** — configuração dos horários disponíveis por dentista e sala.
- **Lista de Espera** — pacientes aguardando vaga.
- **Retorno** — controle de pacientes com retorno pendente.
- **Comunicações** — envio de lembretes/confirmações (WhatsApp ou SMS futuro).

5.3 Financeiro

- **Orçamentos** — criação e acompanhamento de orçamentos por paciente/tratamento.
- **Financeiro** — lançamentos de receitas e despesas; contas a receber; relatório de caixa.

5.4 Materiais

- **Materiais** — cadastro e controle de estoque de insumos.
- **Esterilização** — registro de ciclos de esterilização de instrumentais.
- **Rastreabilidade** — histórico de uso de instrumentais por consulta.

5.5 Relatórios

- **Relatórios** — produtividade por dentista, ocupação de agenda, receita por período, consumo de materiais.

5.6 Administração

- **Dentistas** — cadastro dos profissionais da clínica.
- **Salas & Unidade** — configuração das salas de atendimento.

- **Usuários** — gestão de usuários do sistema e perfis de acesso.
 - **Auditoria** — log de ações relevantes no sistema.
 - **Backup** — exportação e importação de dados (JSON).
-

6. Estrutura de Dados (Entidades Principais)

AppUser (Usuário do Sistema)

- `id` (UUID)
- `email` (String, único)
- `password_hash` (String — nunca armazenado em texto plano)
- `name` (String)
- `role` (Enum: ADMIN, SECRETARIA, DENTISTA)
- `active` (Boolean)
- `created_at` / `updated_at`

Dentista

- `id` (UUID)
- `name` (String)
- `cro` (String — registro profissional)
- `specialty` (String)
- `active` (Boolean)
- `created_at` / `updated_at`

Sala

- `id` (UUID)
- `name` (String — ex.: "Sala 1", "Sala 2")
- `active` (Boolean)

Paciente

- `id` (UUID)
- `name` (String)
- `cpf` (String, único)
- `birth_date` (Date)
- `phone` (String)
- `email` (String)
- `address` (String)
- `notes` (Text — observações gerais)
- `active` (Boolean)
- `created_at` / `updated_at` / `deleted_at`

Consulta

- `id` (UUID)
- `patient_id` (UUID → Paciente)
- `dentist_id` (UUID → Dentista)
- `room_id` (UUID → Sala)
- `scheduled_at` (Timestamp — data e hora agendadas)
- `duration_minutes` (Integer)
- `status` (Enum: AGENDADA, CONFIRMADA, EM_ATENDIMENTO, CONCLUIDA, CANCELADA, NAO_COMPARECEU)
- `type` (String — ex.: "Limpeza", "Retorno", "Avaliação da restauração")
- `notes` (Text — anotações da consulta)
- `created_at` / `updated_at` / `deleted_at`

Prontuário (ProntuarioEntry)

Cada entrada representa um registro clínico dentro de uma consulta. - `id` (UUID) - `patient_id` (UUID → Paciente) - `consulta_id` (UUID → Consulta, nullable — pode haver anotação fora de consulta) - `dentist_id` (UUID → Dentista) - `entry_date` (Date) - `description` (Text) - `created_at`

Orçamento

- `id` (UUID)
- `patient_id` (UUID → Paciente)
- `dentist_id` (UUID → Dentista)
- `status` (Enum: PENDENTE, APROVADO, REPROVADO, CONCLUIDO)
- `total_value` (Decimal)
- `notes` (Text)
- `created_at` / `updated_at`

OrcamentoItem

- `id` (UUID)
- `orcamento_id` (UUID → Orçamento)
- `description` (String — procedimento)
- `quantity` (Integer)
- `unit_price` (Decimal)

LancamentoFinanceiro

- `id` (UUID)
- `type` (Enum: RECEITA, DESPESA)
- `patient_id` (UUID → Paciente, nullable)
- `orcamento_id` (UUID → Orçamento, nullable)
- `description` (String)
- `amount` (Decimal)
- `due_date` (Date)
- `paid_at` (Date, nullable — null = em aberto)
- `payment_method` (String — ex.: "Dinheiro", "Cartão", "PIX")
- `created_at` / `updated_at`

Material

- `id` (UUID)
- `name` (String)
- `unit` (String — ex.: "unidade", "caixa", "ml")
- `stock_quantity` (Decimal)
- `min_stock` (Decimal — quantidade mínima para alerta)
- `active` (Boolean)
- `created_at` / `updated_at`

CicloEsterilizacao

- `id` (UUID)
- `cycle_date` (Date)
- `responsible_id` (UUID → AppUser)
- `autoclave_id` (String — identificação do equipamento)
- `notes` (Text)
- `created_at`

InstrumentalRastreabilidade

Vincula um instrumental a um ciclo de esterilização e a uma consulta. - `id` (UUID) -

`instrumental_name` (String) - `ciclo_id` (UUID → CicloEsterilizacao) - `consulta_id` (UUID → Consulta, nullable) - `created_at`

7. Autenticação e Segurança

7.1 Modelo de Autenticação (Access Token + Refresh Token)

- `POST /api/auth/login` — autentica e retorna:
- **Access Token** — JWT de vida curta (60 minutos), enviado no header `Authorization: Bearer <token>` em todas as requisições às rotas de negócio.
- **Refresh Token** — string opaca de vida longa (30 dias), persistida no banco (tabela `refresh_tokens`). No frontend, armazenada em cookie `HttpOnly` (não acessível por

JavaScript — proteção contra XSS).

- `POST /api/auth/refresh` — renova o Access Token usando o Refresh Token. Retorna 401 se inválido/expirado.
- `POST /api/auth/logout` — revoga o Refresh Token no banco.

Regra de ouro: o backend extrai o `user_id` sempre do contexto de segurança (token validado), nunca de um campo vindo no corpo da requisição. Isso impede que um usuário tente acessar dados de outro informando um `user_id` diferente no payload.

7.2 Refresh Token — por que cookie HttpOnly e não localStorage

Num SPA web, o `localStorage` é acessível por qualquer JavaScript da página — um ataque XSS poderia roubar o token. O cookie `HttpOnly` resolve isso: o browser envia o cookie automaticamente em cada requisição, mas o JavaScript da página não consegue lê-lo.

7.3 Perfis de Acesso (Roles)

Perfil	Permissões
ADMIN	Acesso total, incluindo Usuários, Auditoria e Backup
SECRETARIA	Agenda, Pacientes, Financeiro, Materiais, Comunicações
DENTISTA	Prontuários dos próprios pacientes, Agenda própria (leitura)

7.4 Renovação Automática de Token (HTTP Interceptor Angular)

1. Interceptor detecta resposta 401.
2. Pausa a requisição que falhou (e concorrentes).
3. Chama `POST /api/auth/refresh`.
4. Sucesso: salva novo Access Token, refaz as requisições pausadas — usuário não percebe nada.
5. Falha: limpa sessão e redireciona para login. Único cenário de logout forçado.

AuthGuard: protege rotas verificando a presença do Refresh Token (via cookie) antes de liberar o acesso — sem chamada de rede.

8. Estratégia de Testes

Adotamos uma abordagem híbrida: **TDD no backend** e **BDD no frontend**.

8.1 Backend (Java / Spring Boot) — TDD

Ciclo Red-Green-Refactor com JUnit. Áreas prioritárias:

- **Serviço de agendamento:** conflito de horários (mesma sala/dentista no mesmo horário), regras de cancelamento.
- **Controle financeiro:** cálculo de valores em aberto, geração de recibo.
- **Controle de estoque:** decremento ao registrar uso, alerta de estoque mínimo.
- **Autenticação:** login, refresh, revogação de token.
- **Persistência:** testes de integração com Testcontainers (PostgreSQL real, não mockado).

8.2 Frontend (Angular) — BDD

Cenários escritos antes da implementação de cada tela. Exemplos:

Agendamento: - *Dado que* a Sala 1 já tem consulta às 09:00, *quando* a secretária tenta agendar outra consulta na Sala 1 às 09:00, *então* o sistema exibe erro de conflito de horário.

Financeiro: - *Dado que* um lançamento está em aberto, *quando* a secretária marca como pago, *então* o `paid_at` é preenchido com a data atual e o lançamento sai da lista de pendências.

Offline: - *Dado que* a conexão foi perdida, *quando* a secretária tenta salvar um novo agendamento, *então* o sistema exibe uma mensagem informando que não é possível salvar sem conexão.

9. Infraestrutura Docker

Backend 100% containerizado para espelhar o ambiente de produção:

- `backend/docker-compose.yml` orquestra dois serviços: PostgreSQL e Spring Boot.

- `backend/.env` define portas externas customizadas (evitar conflito com outros projetos locais):
 - API: porta externa `8086` → `8080` interno.
 - Banco: porta externa `5441` → `5432` interno.
 - Produção: deploy na VPS Hostinger com Docker Compose.
-

10. Roadmap Inicial (MVP)

Fase 1 — Base estrutural

1. Setup do workspace: pastas `frontend/` (Angular) e `backend/` (Spring Boot + docker-compose).
2. Autenticação: login, JWT, refresh token, AuthGuard, interceptor.
3. Cadastro de Pacientes: CRUD completo.
4. Cadastro de Dentistas e Salas.

Fase 2 — Atendimento

1. Agenda: visualização por dia e semana, criação/edição de consultas, validação de conflito de horário.
2. Dashboard: cards de resumo do dia.
3. Prontuário: registro de entradas clínicas por paciente.
4. Pendências e Retornos.

Fase 3 — Financeiro e Materiais

1. Orçamentos: criação, itens, aprovação.
2. Financeiro: lançamentos, contas a receber, relatório de caixa.
3. Materiais: cadastro, estoque, alertas de mínimo.
4. Esterilização e Rastreabilidade.

Fase 4 — Relatórios e Administração

1. Relatórios gerenciais (produtividade, ocupação, receita, consumo).
 2. Gestão de Usuários e perfis.
 3. Auditoria de ações.
 4. Backup (exportar/importar JSON).
-

11. Observações Técnicas

- **IDs gerados pelo backend:** como não há gravação offline, não há necessidade de UUID gerado no frontend. O backend é a única fonte de verdade desde o primeiro registro.
- **Soft delete:** entidades principais nunca são apagadas fisicamente do banco — recebem `deleted_at`. Isso preserva integridade referencial (uma consulta deletada ainda pode ser referenciada num lançamento financeiro) e permite auditoria histórica.
- **Dados de saúde — LGPD:** prontuários e dados clínicos são dados sensíveis sob a LGPD. O sistema deve registrar em log quem acessou cada prontuário (campo de auditoria) e não expor esses dados em listagens gerais — apenas no acesso direto ao prontuário do paciente.